

SECURE CODING REVIEW REPORT

CodeAlpha Cyber Security Internship

Task 3: Secure Coding Review

Submitted By:

Gandamalla Likitha

Date:

August 2026

Page 2 – Table of Contents

- 1.Introduction
- 2.Objectives
- 3.Application Overview
- 4.Vulnerability Analysis
- 5.Security Improvements
- 6.Risk Assessment
- 7.Best Practices
- 8.Conclusion
- 9.References

Introduction

Secure coding is the practice of writing software that protects applications from security vulnerabilities and cyber threats. Poor coding practices can lead to unauthorized access, data breaches, and application compromise.

This review evaluates a Python-based login application to identify security weaknesses and recommend secure coding solutions.

Objectives

The objectives of this review are:

- Identify security vulnerabilities
- Analyze associated risks
- Demonstrate secure coding practices
- Improve application security
- Promote secure software development

Application Description

The application is a simple login system developed using Python and SQLite.

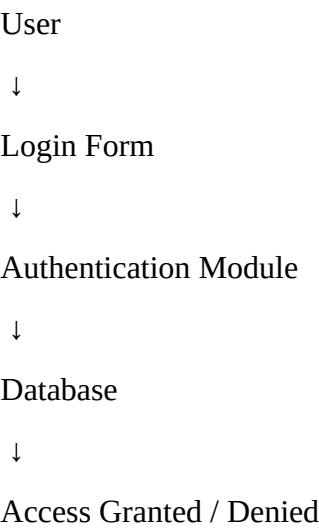
Features

- User Registration
- User Login
- Credential Verification

Technology Stack:

Compoent	Technology
Language	Python
Database	SQLite
Security	bcrypt

Workflow:



Vulnerability 1

Hardcoded Credentials

Description

Sensitive credentials are stored directly in source code.

Vulnerable Code

```
ADMIN_USERNAME = "admin"  
ADMIN_PASSWORD = "admin123"
```

Impact

- Credential exposure
- Unauthorized access
- Security breach

Severity

HIGH

Recommendation

Store credentials using environment variables.

Vulnerability 2

SQL Injection

Description

User input is directly concatenated into SQL queries.

Vulnerable Code

```
query = "SELECT * FROM users WHERE username='"  
+ username + "'"
```

Example Attack

```
' OR '1'='1
```

Impact

- Authentication bypass
- Data theft
- Database manipulation

Severity

HIGH

Secure Code

```
cursor.execute(  
    "SELECT * FROM users WHERE username=?",  
    (username, )  
)
```

Vulnerability 3

Plain Text Password Storage

Description

Passwords are stored without hashing.

Vulnerable Example

```
password = "password123"
```

Impact

- Password disclosure
- Account compromise

Severity

HIGH

Secure Code

```
bcrypt.hashpw(  
    password.encode(),  
    bcrypt.gensalt()  
)
```


Vulnerability 4 & 5

Missing Input Validation

Impact

- Unexpected application behavior
- Security risks

Secure Example

```
if len(username) > 30:  
    print("Invalid Username")
```

Detailed Error Messages

Vulnerable Example

```
print("Database Error:", e)
```

Impact

Information disclosure.

Secure Example:

```
print("An error occurred.")
```

Severity

MEDIUM

Vulnerability 6

Weak Password Policy

Description

Weak passwords are accepted without restrictions.

Risk

- Easier password guessing
- Brute-force attacks

Secure Example

```
if len(password) < 8:  
    print("Password must contain at least 8 characters.")
```

Severity

MEDIUM

Security Improvements

Implemented Security Controls

Parameterized Queries

Prevents SQL Injection attacks.

Password Hashing

Protects stored passwords.

Input Validation

Ensures valid user input.

Secure Error Handling

Prevents information leakage.

Password Policy

Enforces stronger passwords.

Risk Assessment

Vulnerability	Severity
Hardcoded Credentials	High
SQL Injection	High
Plain Text Password Storage	High
Missing Input Validation	Medium
Detailed Error Messages	Medium
Weak Password Policy	Medium

Overall Risk Rating

HIGH

Secure Coding Best Practices

- Input Validation
- Password Hashing
- Parameterized Queries
- Secure Authentication
- Least Privilege Principle
- Secure Error Handling
- Security Testing
- Code Reviews

Conclusion

The review identified multiple security weaknesses in the login application. Vulnerabilities such as SQL Injection, hardcoded credentials, and plain text password storage pose significant risks.

By implementing secure coding techniques including parameterized queries, password hashing, input validation, and secure error handling, the security posture of the application is greatly improved.

References

1. OWASP Top 10
2. OWASP Secure Coding Practices
3. Python Documentation
4. SQLite Documentation
5. bcrypt Documentation
6. NIST Cybersecurity Framework